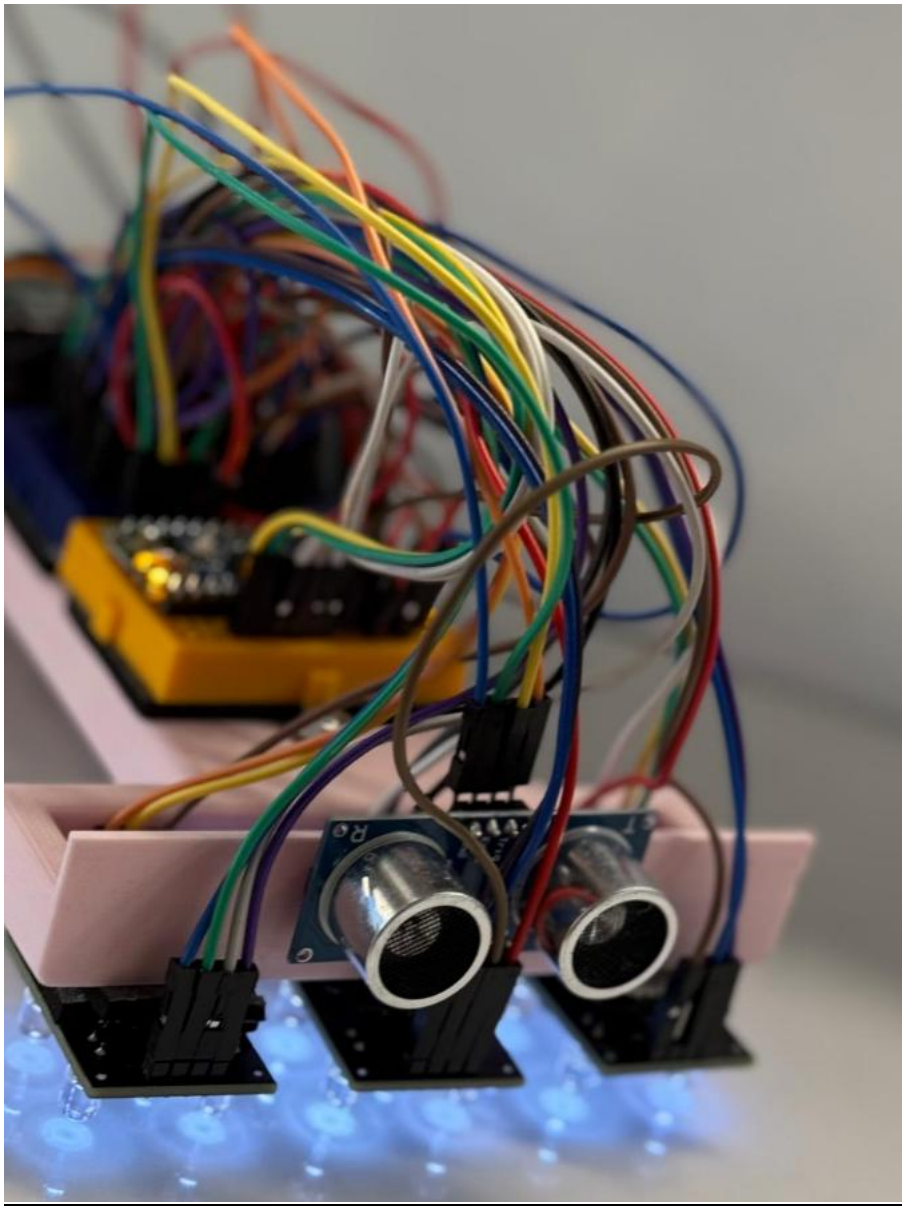


Line Follower Robot Design and Build Project (Group 9)

Sajid Masood, 230538579

Hisham Bakhsh, 231156930

Vadim Busuioc, 230515086

Wahab Faizi, 230292154

Kyrullos Aziz, 230477474

Abstract

Intelligent transportation systems (ITS) are advanced applications which, without embodying intelligence as such, aim to provide innovative services relating to different modes of transport and traffic management and enable various users to be better informed and make safer, more coordinated, and 'smarter' use of transport networks, this then progresses into self-driving cars which have been of great interest to big tech companies like Google. To break it down, we can think of these cars as robots simply trying to follow instructions as well as trying to learn and adapt to road environments. This all starts with following a line.

The primary focus of this project was to make a Line Follower Robot which can follow and detect black line on a sheet of glossy paper as well as having obstacle detection and avoidance system.

The main methodology this project will follow will be:

- Creating and designing an appropriate chassis that suits the robot with the addition of suitable components such as IR or RGB sensors, motos, motor drivers and microcontroller.
- Integration and interfacing of components and getting them to perform their desired task
- Implementation of different control systems such as PID and Bang-Bang control and using the one that maintains the best trajectory of the line
- Troubleshooting and testing with Arduino Nano Every microcontroller and interfacing it with other components
- Adding object and obstacle avoidance by integrating the ultrasonic sensor with our microcontroller.

We were given two different types of tracks, one being an oval and another being the Silverstone Circuit which consisted of many sharp and aggressive bends. The tracks all had black lines on a glossy surface and was expected for the robot to follow the lines however many issues were encountered during this. The following sections will record our progress and explore the specifics in detail.

Table of Contents

1. Introduction

This project involves the construction and development of a line-following robot. Typically, on a black and white surface with the ability to autonomously detect and follow the marked path. Line-following robots play a crucial role in the fields of robotics and automation, frequently utilized in warehouses, manufacturing facilities, and educational settings to illustrate essential concepts in sensing, control, and embedded systems.

The main objective of this project is to construct a robot that can precisely and accurately follow a predetermined path. Microcontrollers are used to process data achieved from sensors to control the movement of the robot. In addition to colour or infrared sensors, an ultrasonic sensor is to be used to detect and avoid obstacles in its path.

The remainder of the report is structured as follows: Firstly, the specifications section outlines the performance and design requirements. Next, the background section provides context and relevant technologies. The design procedures provide a basis for the software and hardware development process. Testing and results present the testing procedure in addition to the outcomes. Evaluation and Discussion offer analysis of the systems behaviours and a summary of the results. Future improvements discuss potential upgrades to be done. Lastly, the conclusion is used to finalize the report.

2. Specification

This project's primary objective is to create an autonomous vehicle that can follow a black line on a white background and avoid obstacles. Because of its ease of use and versatility, this prototype is primarily used in automated transportation and instructional contexts, providing a hands-on example of simplified autonomous navigation approaches.

The functional block diagram for the system, which shows the key parts of our car and how they work together, is shown below (Figure 1). By detecting colour contrasts on shiny surfaces, RGB colour sensors are able to recognise the defined path and provide sensor data straight to the Arduino microcontroller (MCU). The MCU determines the motor driver's instructions by interpreting the RGB sensor readings and processing them using a condensed algorithm, such as PID or Bang-Bang. In order to detect obstacles, the ultrasonic sensor also continuously measures distances ahead of the car. The sensor alerts the MCU to stop or reroute the vehicle when it detects obstacles within a predetermined range.

Two DC motors that power the wheels are controlled by motor drivers, which are in turn controlled by the Arduino microcontroller. The power supply, which consists of a typical battery source, provides the necessary operational voltage and current to the complete system.

Here is our initial system block diagram:

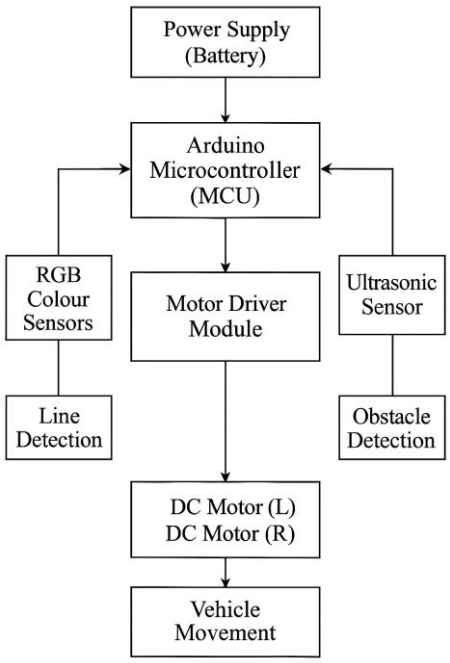


Figure 1: Initial System block diagram

Key Specifications and Requirements:

- **Line-Following Capability:**
  The vehicle must use appropriate sensors to precisely detect and follow a designated path on reflected surfaces.

- **Obstacle Detection and Avoidance:**
  Using ultrasonic sensors, the system should accurately identify objects within a predetermined range before carrying out the required avoidance actions.

- **Control Algorithm:**
  For accurate steering and speed changes, the car needs to use an appropriate control method, such as PID or Bang-Bang control algorithms.

- **Chassis Design and Structural Integrity:**
  It is important to create a robust chassis design that considers optimal manoeuvrability, sensor integration simplicity, and structural stability.

- **Budget Limitation:**
  The entire cost of the project, including all parts and production procedures, cannot be more than £100.

- **Operational Ease and Reliability:**
  With minimal input from the user, the vehicle should be able to carry out its tasks independently with a high level of reliability, stability, and consistency across a variety of track designs.

3. Background

3.1. Arduino Microcontrollers (Uno Vs Nano)

Arduino microcontrollers, such as the Uno and Nano, are commonly used in robotics projects. This is due to their affordability, versatility and the fact that they are very easily accessible. Both the Uno and Nano use the ATmega328P microcontroller, therefore they both offer similar I/O pins, processing power and programming capability [1]. However, they differ in their physical size and energy consumption. The Nano has smaller dimensions than the Uno whilst also having a higher power efficiency [2]. Hence, the Nano is more suitable than the Uno for robotic applications with limited space where energy conservation is a crucial aspect.

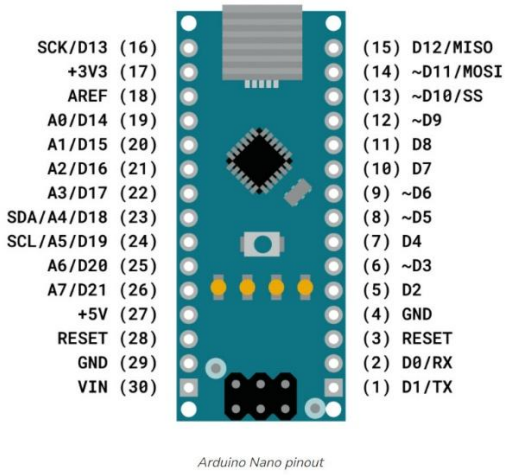


Figure 2: Arduino Nano pinout map

3.2. Sensors

RGB and Colour Sensors:

RGB colour sensors, like the TCS3200, work by measuring the intensity of reflected light at wavelengths (red, green and blue). Therefore, these sensors would be appropriate for our robot since they can detect changes in surface reflectance and our vehicle would be driving on a glossy reflective surface [3]. Although RGB sensors give precise readings, they need to be carefully calibrated and have their lighting conditions adjusted to gain the best results [4].

IR Sensors:

Infrared (IR) sensors are also commonly used in autonomous line-following robots due to their ability to detect changes in surface reflectivity, guiding the robot along a path marked by a colour contrast between the line its following and its background. The

sensors operate by emitting IR light and measuring its reflection, and can be configured in many ways, from simple single sensors which are ideal for straightforward paths, to complex arrays for navigating more difficult paths. However, their effectiveness is limited to a glossy or highly reflective surface where a colour sensor may perform better.

Ultrasonic sensors:

Ultrasonic sensors, such as the HC-SR04, operate based on the time-of-flight principle. They send out ultrasonic pulses to calculate the distance to obstacles, by timing the time it takes for echoes to return. This time can then be used to calculate the distance, which can be used in the code. Ultrasonic sensors are crucial for avoiding obstacles. This is because they work in a range of lighting conditions and are highly accurate in identifying obstacles in the robot's path.

3.3. Motor Drivers (L298N vs. TB6612FNG)

Motor drivers are necessary in our project to interface low-current Arduino signals with higher-current motors. Due to its durability and its ability to manage large voltages and currents, the L298N is frequently used. However, because of its BJT transistor architecture, it experiences severe power loss, so it may not be ideal for our robot [5]. Alternatively, the TB6612FNG uses MOSFET transistors, which significantly increases efficiency and lower voltage dips, therefore making it more suitable for battery-powered robots that need to use power efficiently [6].

3.4. Motors

There are 3 motor types taken into consideration for robotics, and these include DC motors, stepper motors, and servo motors. DC motors tend to be recommended for small robots, due to their low cost, ease of speed with PWM (Pulse Width Modulation), and simplicity [7]. Although they provide accurate control, stepper motors aren't suitable for mobile robotics that need to move continuously as they are much slower and less efficient than DC motors [8]. Additionally, although servo motors are great for controlling the position of the vehicle precisely, they have limited rotational range, unless modified for continuous rotation, which makes it difficult to implement them in simple drive mechanisms [9].

3.5. Chassis Design (Prefabricated Vs Custom-Built)

Since pre-made chassis are produced in large quantities, they are easy to assemble, convenient and tend to be cheaper than making a chassis. However, a custom-built chassis allows for a much greater control over the design of the chassis, improved mechanical stability, and more flexibility with customized component integration, allowing you to adjust positions of components in order to prioritize efficiency. Though a custom-built chassis has its benefits, its drawbacks must also be taken into consideration. These include a much longer time spent in design work, fabrication tools, and maybe significantly greater prices than a pre-made chassis if materials are outsourced [10].

4. Design Procedures

4.1. Chassis Deep Research

The design and structure of the chassis greatly depends on the size of all the components we are using, therefore we can make a chassis that it is not too large or small and appropriate size for the tracks it will be tested on. Additionally, accurately sizing these elements will allow for a chassis design suited to the track that will be efficient with optimal performance. Below the dimensions of all the electrical components being used will be listed below:

- **TB6612FNG Dual Motor Driver**: 20 mm x 15 mm
- **Arduino Nano**: 45 mm x 18 mm
- **N20 Motors**: Length 32 mm, Diameter 12 mm
- **QTR-8RC IR Sensor Array**: 75 mm x 13 mm
- **9V Battery**: 50mm x 25mm

There will also be two non-electrical components used which will be the castor ball and side wheels for going forward and back, dimensions attached below:

- **Iron Frame and Nylon Castor Ball:**
    - Overall Width: 48mm
    - Base Length: 38mm
    - Base Height: 22mm
    - Ball Diameter: 14mm
    - Mounting Hole Diameter: 4mm
- **N20 motor Rubber Wheels:**
    - Tyre Diameter: 44mm
    - Tyre thickness: 18mm

With all the dimensions being established now, this gives us a robust foundation from which we can develop and conceptualize the structure and design of the chassis. Not only will we be able to develop a spatial arrangement but also ensures a proximate fit. This makes us ask why is a snug/proximate fit so important? With a small track provided to use this makes it essential to have a compact robot design for it to navigate sharp

bends as well ensuring high standard manoeuvrability and overall better performance. Ultimately, chassis structure is something that needs to be considered carefully as this can either cause issues further in the project and can lead to changing chassis or adjusting.

Selecting a Chassis Design:

Creating a line robot which will follow an identical copy track of the famous Silverstone Circuit then will be ideal to take some inspiration from existing racing vehicle chassis such Go-karts or even formula one racing vehicles. The chassis from these motor sports have already undergone a lot of development and they have been made to the best of ability excelling in things like rigidity and strength, Balance and weight distribution, Integration of Systems (adequate provisions for integration of essential systems so in our case the Microcontroller, Motor Driver etc). Even though, we don't have a big focus on speed and winning races but to just follow the line it is still vital to have a chassis with all these features incorporated so our robot can benefit from advanced engineering practices that enhance performance and reliability.

Typical Racing-Kart Chassis:

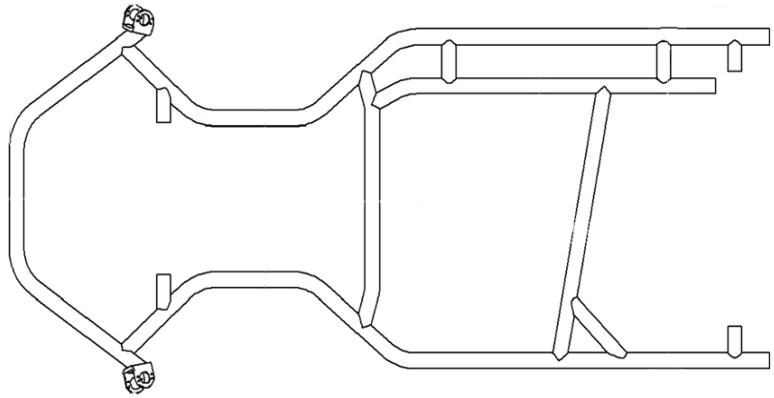


Figure 3: Go-Kart Chassis

The first things that stands out in this design is that the right-hand side (rear) of chassis is much bigger for weight distribution so increases stability and balance.

With a wider rear most likely wheels will also be placed there so better traction and handling.

Having a narrow midsection for better agility and capable of sharp bends as well as being a centralised component layout area.

What will we incorporate from this design to our chassis design?

1) Having a wider and larger rear will be very beneficial for my design so this will be incorporated into my design.
   Why: As mentioned before a wider rear ensures stability and traction, makes sures that the robot remains balanced, especially when taking a sharp turn, preventing it from tipping over. With a wider rear we can also accommodate all the component's especially the larger ones like the Arduino and battery.
2) A narrow midsection will also be used for my chassis.

Why: Once again allows better agility but can be used for more narrow components like a battery.

3) Overall, a design like this will be used but a more angular architecture design for a more aesthetic appeal as well as rigid structure.

My Design and What I have added:

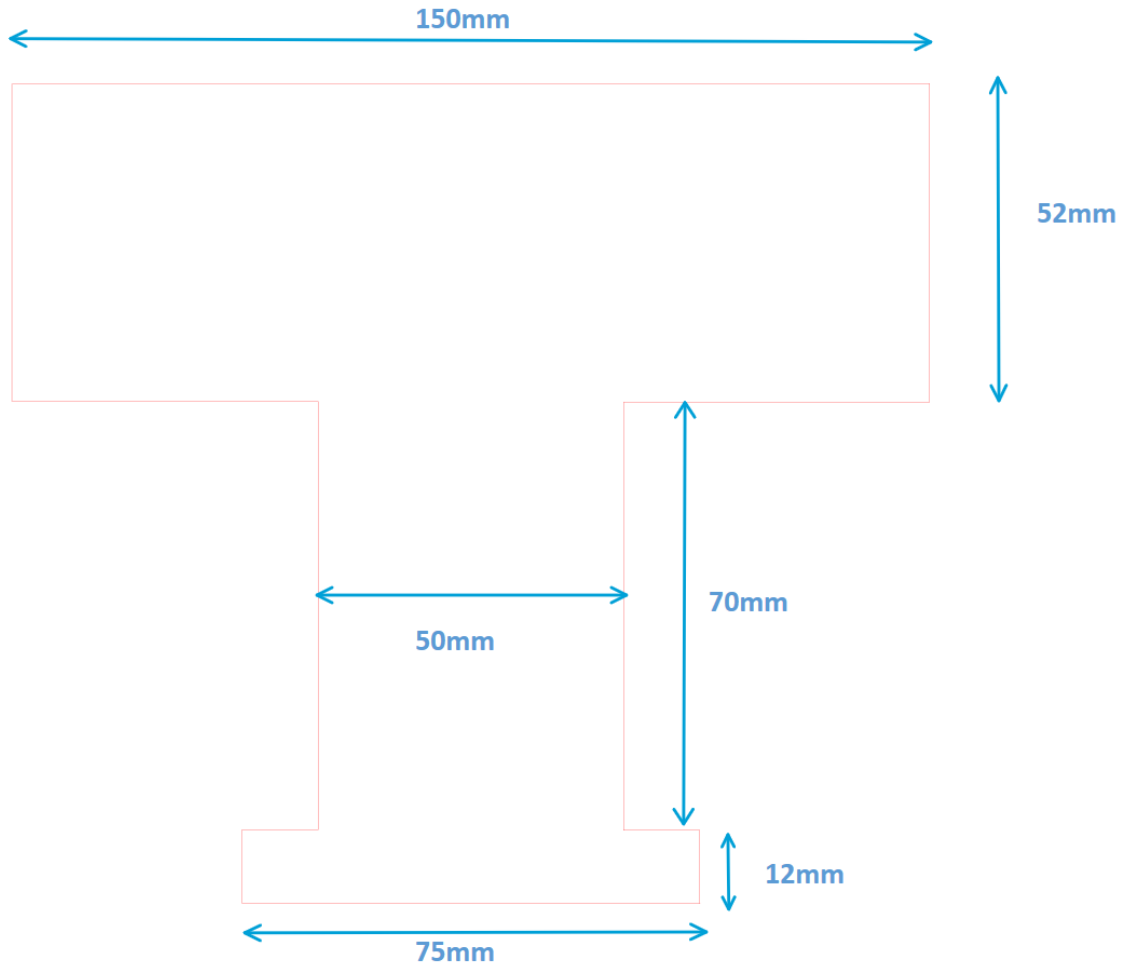


Figure 4: 2D Designed Chassis

This above design was made on Inkscape a software which is compatible with laser cutters allowing me to bring my design to life. Previously, detailing the features inspired from go-kart design, below I will list the features I add myself and adjust from go-kart design:

- A 75mm x 12mm section right at the front, this section is extremely important because it will accommodate the QTR-8RC sensor which will detect the lines and send message to Arduino which then control the motors.

- Made midsection 50mm, with only an IR sensor and lightweight castor wheel at the front and all the other components at the back rear end there can be an imbalance in weight which can affect movement of robot, therefore this 50mm midsection will accommodate some components for more weight distribution.
- Additionally, more components in the midsection means more centrally located mass giving us better manoeuvrability also it allows for more simplified wiring layout with this new centre focus, reducing lengths of connections.
- I will demonstrate all these changes in the image below **'Final Laser Cut'.**

Final Laser cut Chassis

In the first image you can see the chassis with two small breadboard which contain the Arduino Nano and Motor Driver connected to a 9V Power Supply (battery).

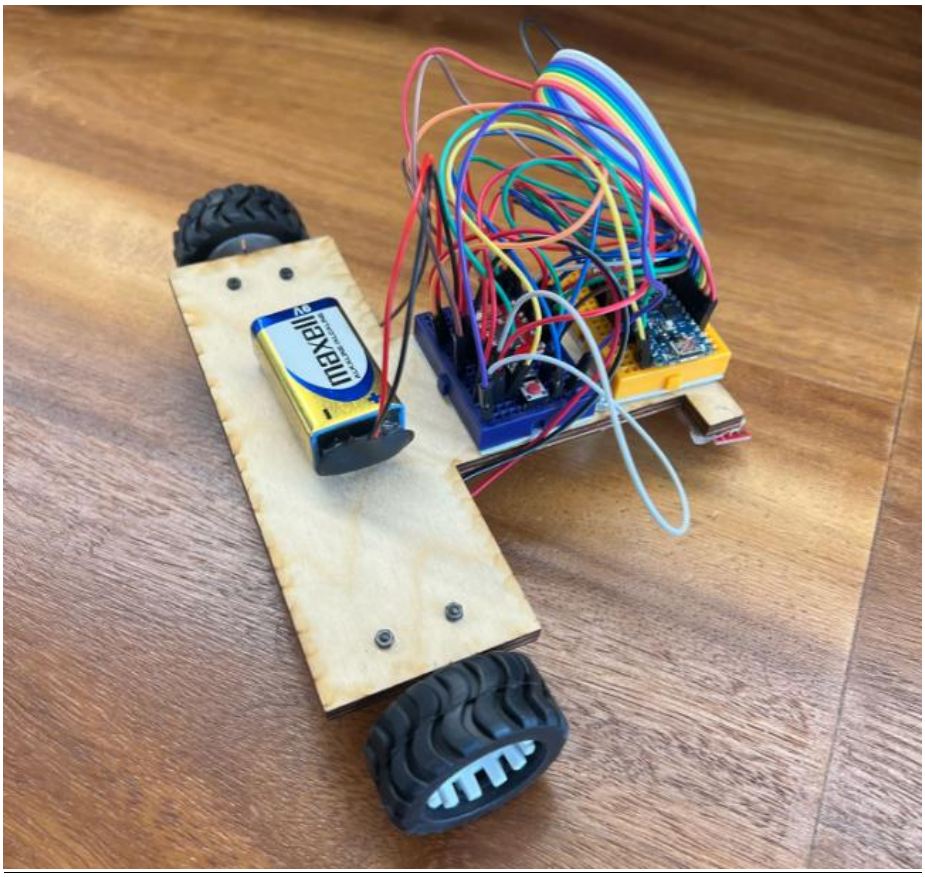


Figure 5: Laser cut chassis mounted with components

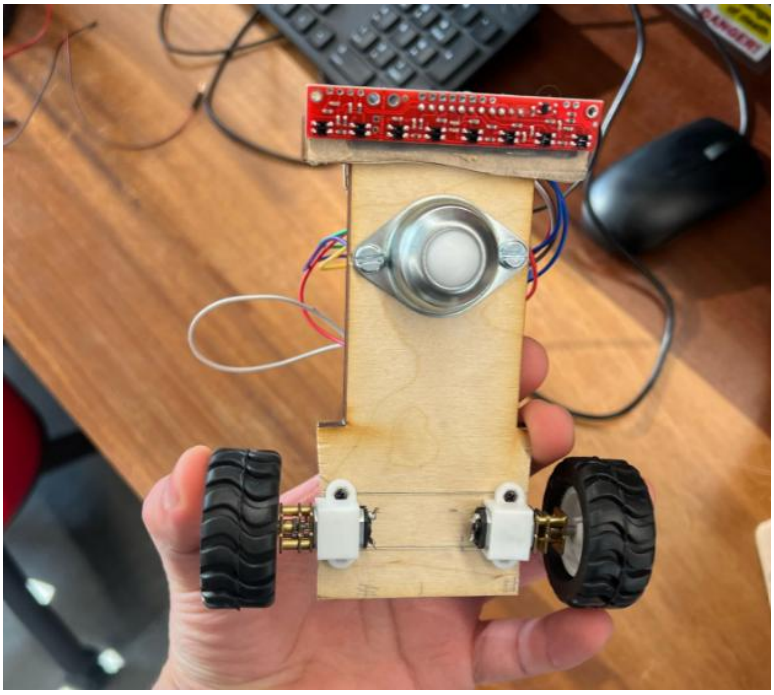


Figure 6: Bottom of the chassis showing castor wheel and IR sensor

4.2. Sensor Selection

Sensor selection was integral to ensure our robot followed the line accordingly and avoided obstacles in its path. Initially, the Pololu QTR-8RC reflectance sensor array was chosen, as it is suited for non-reflective surfaces, and used the PID control algorithm. This combination produced accurate results on non-reflective surfaces.

Pololu QTR-8RC's impressive results on matted surfaces did not transfer when attempting it on a glossy surface. It showed significant limitations in its inability to distinguish between IR signals and ambient lighting. These erratic and unreliable readings led to the switch to TCS32000 color sensors, which would be able to compact the surface's reflectivity.

TCS3200 contains an 8x8 array of photodiodes that work as a programmable color light-to-frequency converter. These RGB sensors allowed the robot to properly detect color differences on a glossy track much more effectively. The transition from IR to RGB sensors led to many substantial redesigns in hardware as well as in the control algorithms. One of which was the transition from PID to Bang-Bang configuration as this simpler method that suited are RGB values better.

Lastly, for obstacle detection, the HC-SR04 ultrasonic sensor was selected. This was due to its easy integration with the Arduino, and its wide availability. It also included an adequate range for detecting objects between 2cm – 400cm and avoiding said objects accordingly.

4.3. Motors and motor driver

The N20 motors have been chosen as they are very compact and provide enough torque for our project. These motors are very good at handling quick acceleration and deceleration which is required for a bang-bang algorithm. This allows it to take very shallow curves as well as acute turns where necessary in a very short time span. Their compact size also allows us to greatly reduce the size of the final chassis and use the extra space for our other components or the battery.

The N20 motors are brushed which means that they can smoothly operate at high speeds whilst not sacrificing torque. Stepper motors, however, cannot operate well at high speeds due to their stepping mechanism. They are more well suited for projects which require small and precise movements.

Compared to other motors, the N20 motors were also very affordable, and we had a wide choice of rpms we could choose from.

To accompany the motors, we have chosen a TB6612FNG motor driver which is a dual H-bridge driver that allows independent control of both motors. It is compatible with the N20 motors as it supports the required high current output of 1.2 A.

The driver is connected to the Arduino Nano which sends it control signals for the motors. The motors then use the control signals to regulate the speed and direction of the motors using PWM (pulse width modulation). The driver includes a PWMA and AIN1/AIN2 to control the left motor and a PWMB and BIN1/BIN2 to control the right motor. The STBY pin enables the driver when HIGH so we have connected the pin across to the VCC pin of the driver which receives the logic voltage, meaning the pin will always be set HIGH. The VM and the GND pins are appropriately connected to the positive and negative terminals of a 9V battery which supplies the needed voltage in order for the motors to operate smoothly.

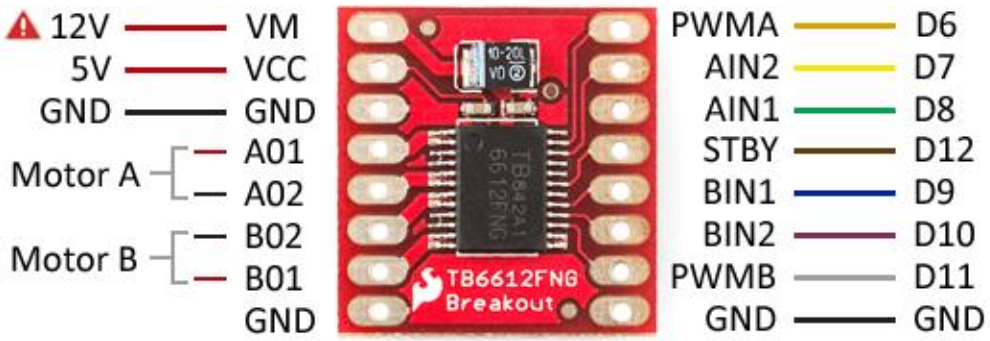


Figure: TB6612FNG motor driver pinout map

4.4. Power Supply

Another important design factor was the choice of power sources. A steady voltage source is necessary to power the microprocessor, motors, motor driver, and sensors for the robot.

In the initial design, a 7.4 LiPo battery was selected. LiPo batteries can deliver the required voltages and current to the motors and other logic components while having a high energy density. However, challenges arose regarding the risk of over-discharge and voltage regulation. These challenges could potentially damage the battery and other circuit components. In addition, the challenges stated the LiPo battery size would require a significant amount of space on the robot. These issues resulted in a search for a better alternative.

In response, to LiPos constraints we pivoted to a 9V battery. This switch of batteries simplified the circuitry and removed the need for any additional charging of hardware, even though it lowers the current capacity. Although this may limit the peak performance of the line-following robot, especially under a high load. It proved the right choice as it enabled a more manageable size and cost-effective solution as the £100 strict budget was taken into consideration.

4.5. PID vs Bang-Bang

The PID line following algorithm is more complex compared to the Bang-bang algorithm because it constantly calculates the error in the position of the robot relative to the centre of the line. Along with the error, three other constants are used in the final calculation which are manually adjusted by the user depending on how we need the robot to react. These three constants are the proportional, the differential and the integral constant.

The proportional constant reacts to how quickly the robot moves away/towards the target. A small proportional constant value will cause lagging turns, while a high proportional constant value will overcompensate for every small change in error and result in uncontrolled oscillation when steering.

The derivational constant helps us reduce these extremes which come with the proportional constant. For example, if error changes too quickly, $\boldsymbol{de/dt}$ will increase, causing the D term to keep the robot from oversteering unnecessarily. A small derivative constant value will cause not have a great enough effect on the motion of the robot, resulting in uncontrolled and jerky oscillations, whilst a high derivative constant value will cause the robot to over-react to changes in error, which will cause very smooth but

delayed responses. Therefore, we need to adjust the value somewhere in the middle by testing it.

The integral constant also helps us have greater control over the motion of the car, however, it is much more sensitive to changes so we can either set it to a very small value or just not use it as it does not contribute as much as the other two constants.

Despite all the cons which arise while using this algorithm, we are not able to use it for our RGB sensor array as we cannot use more than four sensors at a time due to their size and price. This algorithm worked very well while using the QTR8-RC sensor array as there were more sensors detecting the changes in the contrast between the line and the background, leading to a more sensitive value for our position and our error. Unfortunately, however, four sensors do not provide enough accuracy to use a PID algorithm.

Therefore, we have decided to use a much simpler, yet more efficient, algorithm known as the bang-bang algorithm.

The bang-bang algorithm uses several functions which call on each other based on what colour each sensor detects. For example, while the left sensor detects black and the middle and right sensor detect white, the car will take a very shallow left turn. That is until it detects a change in the colour detected by each sensor, at which point it will call on another function within the code and execute it while the values hold true.

Unlike PID, which always updates the motor speeds with any small change in sensor values, the bang-bang algorithm only acts when the sensor values are either smaller or greater than a set threshold upon which it calls certain functions which contain pre-determined motor speeds.

4.6. Architecture and Block Diagram (schematic)

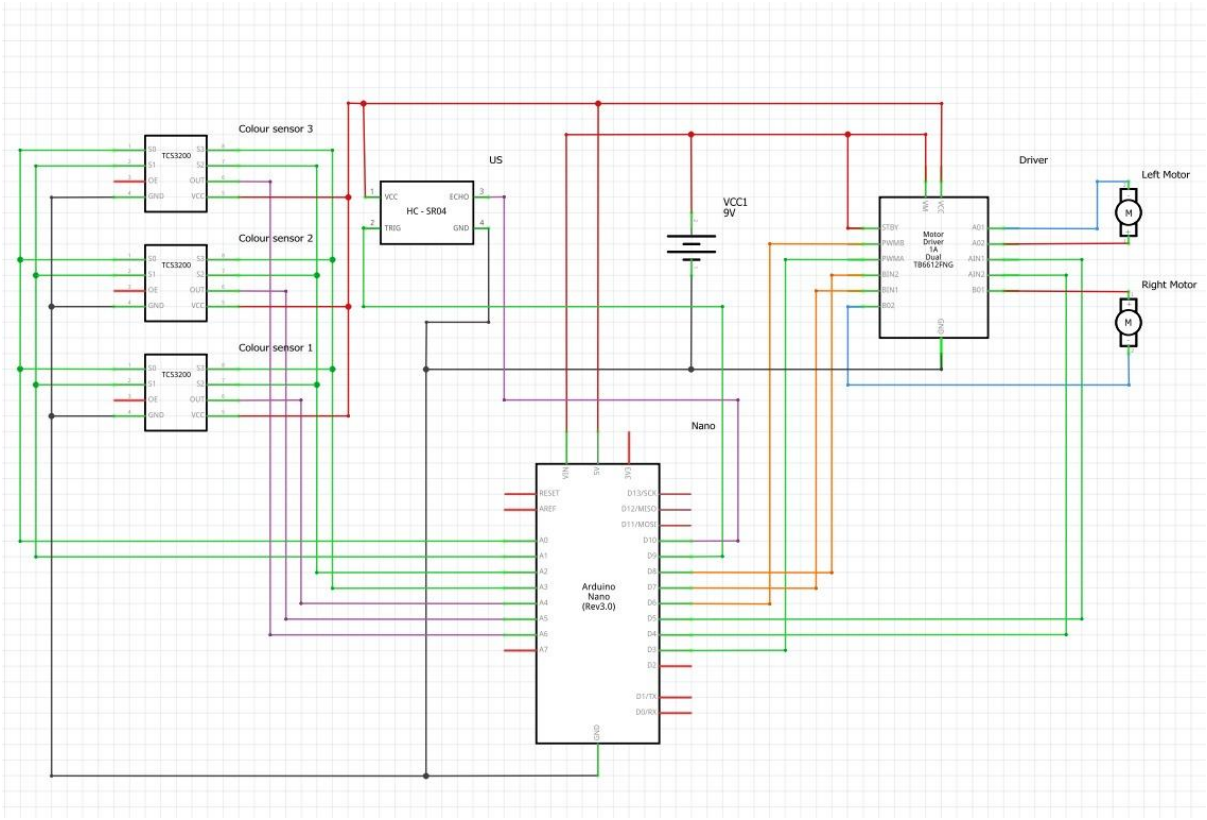


Figure 6: Shows the block Diagram for our robot components

4.7. Initial Code Analysis

Below I will attach screenshots of different parts of our code from the INO fil, which is written in the Arduino IDE 2.3., each screenshot will have a detailed explanation and analysis next to it and will expand on from the comments that are present in the code.

```
1    #include <QTRSensors.h>
```

This line includes the **QTRSensors** library into the code, which will the provide the necessary functions for the QTR sensor to operate and detect the line.

```
3   //Line Sensor Properties
4   QTRSensors qtr;
5   const uint8_t SensorCount = 8;
6   uint16_t sensorValues[SensorCount];
```

The object **qtr** is created from the **QTRSensors** class which will be used for read data from the QTR sensor. In the next line, a constant called **sensorcount** is defined which is set to 8 due a QTR-8RC sensor having 8 arrays. The array **sensorValues** will store readings from each of the 8 sensors each being a 16-bit integer.

```
8   //Motor Driver Properties
9   // Motor A
10  const int PWMA = 3;
11  const int AIN1 = 5;
12  const int AIN2 = 4;
13  // Motor B
14  const int PWMB = 6;
15  const int BIN1 = 7;
16  const int BIN2 = 8;
```

This section defines constants so we can control the motor driver for our robot, where specific Arduino pins are assigned to control the two motors. For example, for **Motor A, pin 3** is assigned as **PWMA** (Pulse Width Modulation) output which is responsible for controlling speed. Another example can be that for **Motor B, Pins 7 and 8** are used to control direction by either making motor go forward

---

```
18  //Need to adjust these constants as we test the robot
19  float Kp = 0.3;
20  float Kd = 0.7;
21  float Ki = 0;
```

This is where PID is integrated and where we set the PID control constraints. **Kp = 0.3**, is for proportional gain controlling reaction to line position errors. **Kd = 0.7**, is for derivative gain, preventing any overshooting. **Ki = 0**, is for integral gain but currently at zero so not being used. These constants are very key and adjust motor depending on its position.

```
23  int last_Error = 0;
24
25  int Calibrate_button = 9;
26  int Start_button = 10;
```

On **line 23**, the last PID error is stored, which is used for calculating the derivative component in next iterations. **Pin** 9 is then assigned to **calibrate_button** to start sensor calibration. **Pin 10** is set to **start_button**, so it starts executing operations.

```
28  void setup()
29  {
30    // Motor A : set all pins as outpu
31    pinMode(AIN1, OUTPUT);
32    pinMode(AIN2, OUTPUT);
33    pinMode(PWMA, OUTPUT);
34
35    // Motor B : set all pins as outpu
36    pinMode(BIN1, OUTPUT);
37    pinMode(BIN2, OUTPUT);
38    pinMode(PWMB, OUTPUT);
```

We start the **setup ()** function where the Arduino pins are connected to motor driver are made to operate as outputs. So, for Motor A, **line 31 and 32** are used to set direction control pin as outputs, so the Arduino can determine its behaviour. **Line 33** is used to configure the speed control pin for motor A as outputs therefore the speed can be adjusted. The same happens for **motor B**. these changes and configuration allows for control of direction and speed of motors based on commands and sensor inputs.

```
40    // Configure the sensors
41    qtr.setTypeRC();
42    qtr.setSensorPins((const uint8_t[]){21,20,19,18,17,16,15,14}, SensorCount);
```

So, on **line 41** is a very important method, it makes sensors to use **RC (resistive-capacitive)** timing mode, the reflectivity is measured depending on the capacitor discharge time. **Line 42** assigns the different Arduino pins to the eight sensors. This is a key step to have proper sensor operation.

```
44    delay(500);
45
46    pinMode(Calibrate_button, INPUT)
47    pinMode(Start_button, INPUT);
```

A delay of **500ms** is implemented just before execution. On **line 46 and 47**, the pins connected to **calibrate_button** and **start_button** are configured as inputs so their states can then be read.

```
49  while (digitalRead(Calibrate_button) == LOW){};
50
51  pinMode(LED_BUILTIN, OUTPUT);
52  digitalWrite(LED_BUILTIN, HIGH); // turn on Arduino's LED to indicate we are in calibration mode
```

The execution is held in a loop until calibrate_button is pressed which will be a LOW signal. Then the Arduino inbuilt LED is set as an output and turned on to indicate the robot is in calibration mode.

```
54    // 2.5 ms RC read timeout (default) * 10 reads per calibrate() call
55    // = ~25 ms per calibrate() call.
56    // Call calibrate() 400 times to make calibration take about 10 seconds.
57    for (uint16_t i = 0; i < 400; i++)
58    {
59      qtr.calibrate();
60    }
61    digitalWrite(LED_BUILTIN, LOW); // turn off Arduino's LED to indicate we are through with calibration
```

In this section, the sensor calibration is performed over 10 second duration, a loop is run 400 times and in each iteration the function **qtr.calibrate()** is called to adjust sensor settings depending on the surface. When calibration is done, the inbuilt Arduino LED is also turned off.

```
63      while (digitalRead(Start_button) == LOW){};
64    }
65
66    void loop() {
67      PID_Control();
68    }
```

The **while** loop is run and will repeatedly check if the **start_button** has been pressed and will only exit when **HIGH** is read. The **void loop ()** calls upon the **PID_control** function and gets results and feedback from these which will be used to adjust the motor speeds so stays on the lines.

```
70    void PID_Control() {
71      uint16_t positionLine = qtr.readLineBlack(sensorValues);
72      int error = 3500 - positionLine;
73
74      for (uint8_t i = 0; i < SensorCount; i++)
75      {
76        Serial.print(sensorValues[i]);
77        Serial.print('\t');
78      }
79      Serial.println(positionLine);
```

The line following logic will be managed in this section of the **PID_control()** function. In line 71, the line position is read by the sensors, returning a value will which be stored in **sensorValues**. On line 72, the error is calculated by subtracting line position value from our desired value of **3500** finding how far the line position is. In the **for loop**, it goes through each sensor value printing it onto **serial monitor**. On line 79 the line position also printed to serial monitor. This is done for amending, also this section is very important as robot behaviour is adjusted to see how line is detected.

```
81    int P = error;
82    int D = error - last_Error; //Could be inverted
83    int I = error + I;
84    last_Error = error;
85
86    int motor_speed_change = P * Kp + D * Kd + I * Ki;
87
88    int motor_speed_A = 125 + motor_speed_change;
89    int motor_speed_B = 125 - motor_speed_change;
```

Once again, we implement PID so for **proportional** the robot correct path by adjusting motor speed directly proportional to the error. For **derivative** term, the rate of change of error. For **Integral** term, it is an accumulation of the past errors, so robot can react to errors in the past and can correct persistent errors. Errors are updates on **line 84**. On **line 86**, all the PID terms are combined so can figure out the speed adjustment needed. Both motor A and B are then adjusted for an accurate alignment with the line.

```
91    if (motor_speed_A > 90) {
92      motor_speed_A = 90;
93    }
94    if (motor_speed_B > 90) {
95      motor_speed_B = 90;
96    }
97    if (motor_speed_A < -175) {
98      motor_speed_A = -175;
99    }
100   if (motor_speed_B < -175) {
101     motor_speed_B = -175;
102   }
103
104   Motor_Control(motor_speed_A, motor_speed_B);
105 }
```

The Motor speeds are limited in this section so for example the speed of motor A is limited to maximum of **90** and minimum of **175** (forward and reverse), same done for motor B. on line 104, the **motor_control** function is called on motor speeds, and this function sets the motor to those speed limits mentioned in the previous lines. The motors are then activated with these new speed controls, this is essential for safe operation of motors and not

---

```
107  void Motor_Control(int speedA, int speedB) {
108    if (speedA < 0) {
109      speedA = 0 - speedA;
110      digitalWrite(AIN1, LOW);
111      digitalWrite(AIN2, HIGH);
112    }
113      else {
114        digitalWrite(AIN1, HIGH);
115        digitalWrite(AIN2, LOW);
116      }
117
118    if (speedB < 0) {
119      speedB = 0 - speedB;
120      digitalWrite(BIN1, LOW);
121      digitalWrite(BIN2, HIGH);
122    }
123      else {
124        digitalWrite(BIN1, HIGH);
125        digitalWrite(BIN2, LOW);
126      }
127
128    analogWrite(PWMA, speedA);
129    analogWrite(PWMB, speedB);
130  }
```

Once again, we are controlling motor direction and speeds depending on inputs. For Motor A if **speedA** is positive or zero then moves forward else it reverses. The same happens for Motor B. Now in terms of speed control the **analogwrite** function applies the PWM signals to the motors and the speed of the motors will be controlled depending on the

With the analysis completed of the proposed code that will be used for the line robot, the main takeaway is that a PID control system will be used to govern this robot. This system adjusts motor speed in response to line position relative to the sensors, additionally setup is a very key function in our code which initializes hardware interfaces for sensors and motors ensuring proper configuration. Furthermore, the loop function is also very vital as it allows the PID control to constantly adjust the motors speeds and direction based on sensor inputs and allowing the robot to make real time adjustments.

5. Testing and Results

5.1. From IR to RGB

At this point, about Week 6 or 7, it was concluded that we had to account for the switch between a matte to glossy surface. We had a robot that fully worked on a matte surface, following even the Silverstone track to perfection. However, as we tried it with a glossy surface, it became unfunctional. This is because light is scattered at glossy surfaces, meaning that the line might be read as white instead of black. This is a problem as IR sensors test for the reflectivity of either black or white. Changing RGB sensors would mean that we couldn't use the existing libraries and had to tweak the code, which would slow our progress. At first, we were reluctant to switch sensors, so we devised many different ways of solving the problem with our existing robot.

Firstly, we attempted the use of polarized lenses, however these didn't solve anything as they are made for visible light and not IR and while theoretically it could work under certain conditions, it was not a practical fix.

We then tried to cover the sensors, this again seemed theoretically sound, however ambient light still made its way through and distorted the colour judging of the IR sensors.

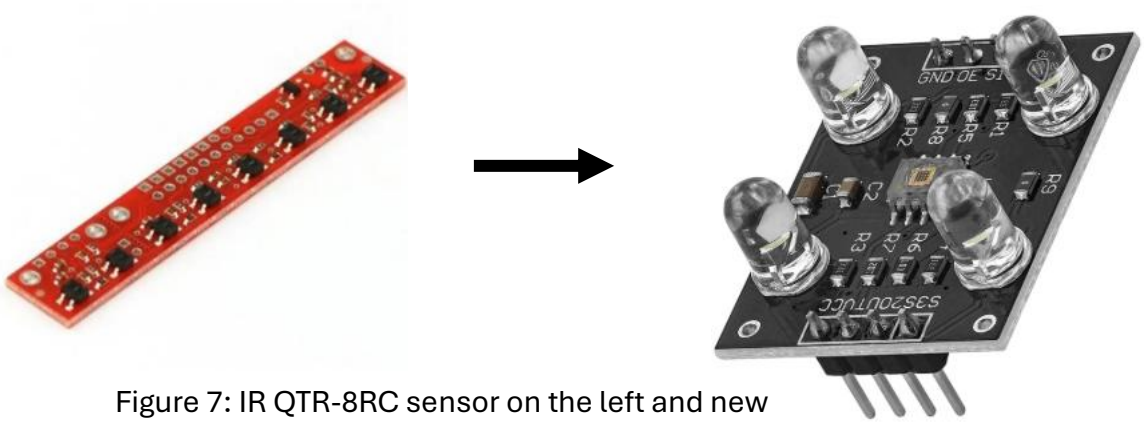


Figure 7: IR QTR-8RC sensor on the left and new RGB TCS3200 sensor on the right

After concluding that these solutions were not practical and why, we then switched to RGB sensors as they were an almost guaranteed way of following the glossy line, as they detect colour not IR radiation. This is done by detecting the intensity of red, green and blue light, converting the intensity from a range of 0-255 which we then map and decide the thresholds for black and white. Our robot then had a significant delay in movement, which we attributed to the sensors. Was potentially buying new sensors the solution?

Well, we tested them, and they were fine. This meant it wasn't necessarily the sensors but the code. Later in the project we concluded that only one colour was sufficient to detect the difference, thus significantly simplifying our code. Simplifying the code heavily decreased the delay, which was another great step for our robot running smoothly.

5.1. Testing and results of Chassis (with issues)

With a wooden laser cut chassis all printed and with all the components wired up as well as the Arduino being programmed with the initial PID control system code, now was the time to start testing and start troubleshooting.

Initial Tests

During the initial tests, the robot was managing to follow straight or nearly straight sections of the track however any places with even little bends it would struggle quite a lot and completely go off track. We figured out a main reason contributing to this issue is that the black track lines were quite narrow relative to the rear width of the chassis and spacing between the wheels make it very difficult for the robot to be able to turn properly(1). Additionally, the compact nature of the track, with its lines all close to each other, also made it difficult. The side wheels were also not level with the castor ball (2), so the robot was slanting downwards making it extremely unesthetic and impractical.

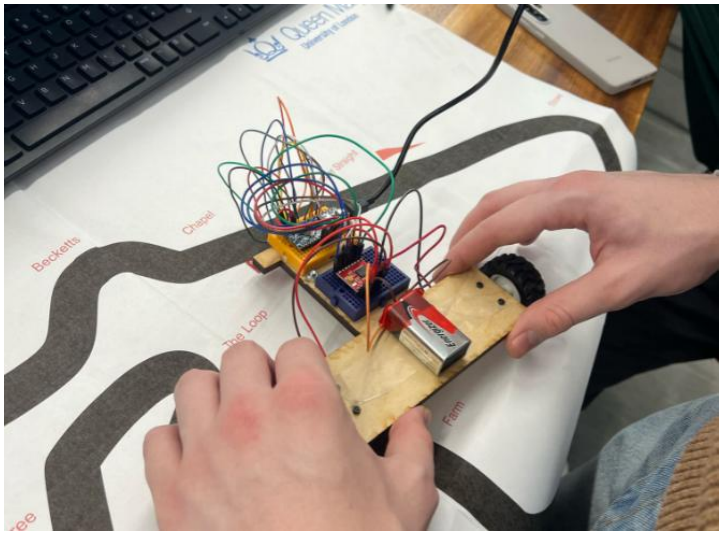


Figure 8: The robot in comparison to the track showing the size difference

In this image on the left, it is clear how large the rear of the chassis is in comparison to the track lines, and it is obvious that any sharp or aggressive bends are unlikely for the robot to successfully complete. There is a lack of wire management (3) which makes it much more difficult to troubleshoot. Later, we will need to incorporate ultrasonic sensor as well for obstacle detection and there is no allocation for

How will we solve these issues I have addressed above?

- Reducing the width of the rear brining the wheels closer together will solve this problem to some extent. (1)

- Making a design such that the castor wheel is on the same level as the rear wheels, this can be done by adding extra thickness to castor wheel area. (2)
- Adding holes to feed wires through for more organised cable management which can help us for future debugging. (3)
- Adding a wall at the front for which we can place an ultrasonic sensor for when we incorporate obstacle detection into the robot. (4)

With all these new required adjustments, we plan to design a new chassis that will be 3D printed. This decision was also influenced by the need for difference in thickness throughout the chassis to align the caster and real wheels correctly, and to also add a small wall to which we can mount the ultrasonic sensor onto. 3D printing and designing gives us this versatility to make such modifications.

New Modified Design

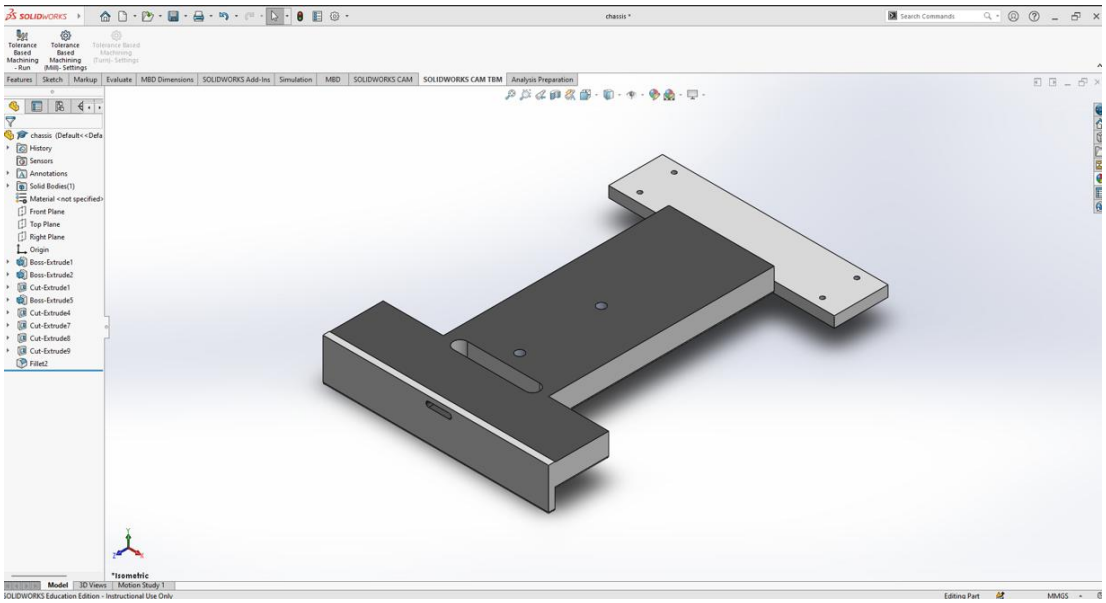


Figure 9: 3D model of the chassis showing its bottom

This 3D design shows the bottom of the chassis, with the large rectangular hole with curved edges to feed wires through, also the wall for ultrasonic sensor and different in thickness (6mm) to level the castor and rear wheels. Below is the 3D printed product, it did not print as well as required due to some loose strings however the overall structure was what we required.

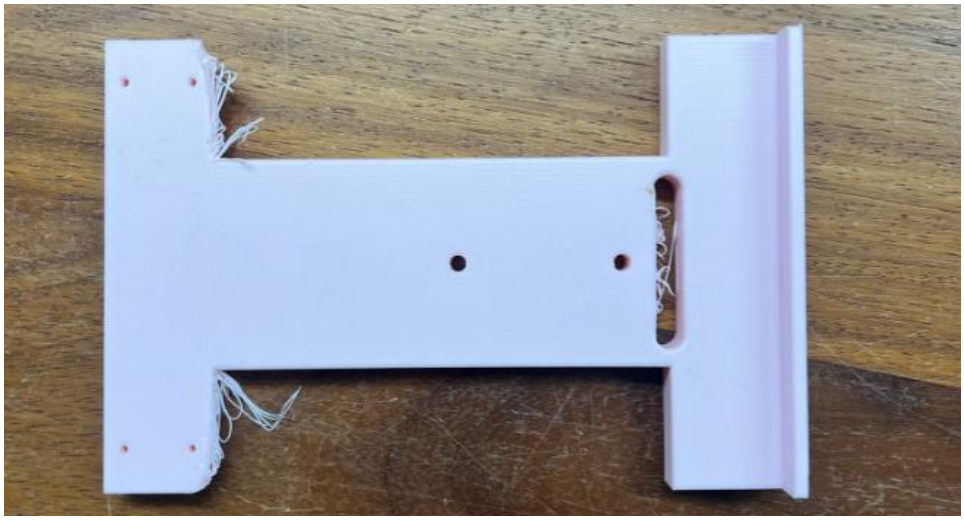


Figure 10: 3D printed chassis

Further testing and problems encountered:

Very quick into this new chassis problems were encountered, firstly it can be seen in the photo above, there were loose strings due top faults whilst it was being 3D printed ruining the aesthetic of the chassis. However, the most major issue was that the screw holes were smaller than anticipated(5) therefore we tried to make them slightly larger by drilling them manually but during this process the rear of the chassis teared and started breaking apart due not having enough strength (6) to withstand the force from these manual hand drills (can be seen in the image below). Finally, the holes made for the wires was too small and just not wires could be fed through it(7). Also now changing the sensors from IR to RGB (specifically the TCS3200), we had to think of way of mounting on these sensors and our current chassis has no allocations or cutout for these new sensors.

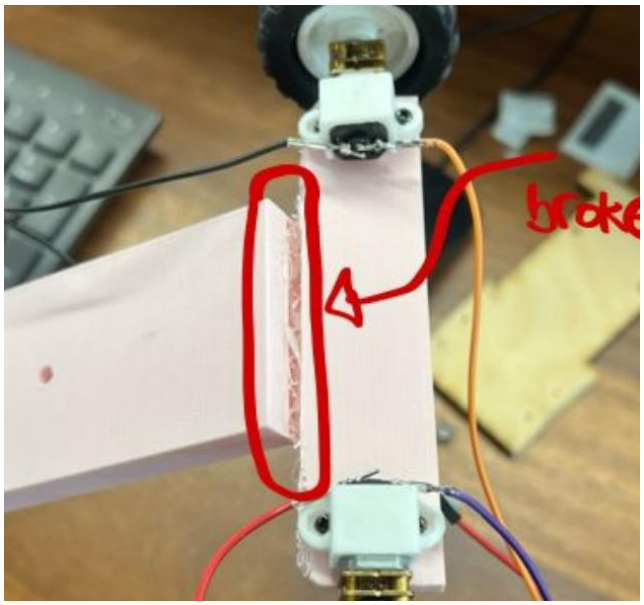


Figure 11: Shows the rear end of chassis breaking

What will be our new solutions?

- Firstly, there is still some tolerance to even reduce the gap between rear wheels even more for enhanced agility. (1)
- Slightly design the screw holes larger on the 3D modelling software (SolidWorks) therefore no need to manually drill into them for correction. (5)
- Adding a ramp at the rear between the two rear wheels for increased stability and no risk of tearing or breaking again. (6)
- Make a much larger hole at the front so wires can be easily fed through it without much complication. This larger cutout at the front will also be used to feed the wires of the new RGB sensors and made wide enough for the RGB sensor to be stuck at the bottom. (7)

I will design again a new chassis, still very similar to this previous design but with some vital modifications.

Modified and FINAL Design

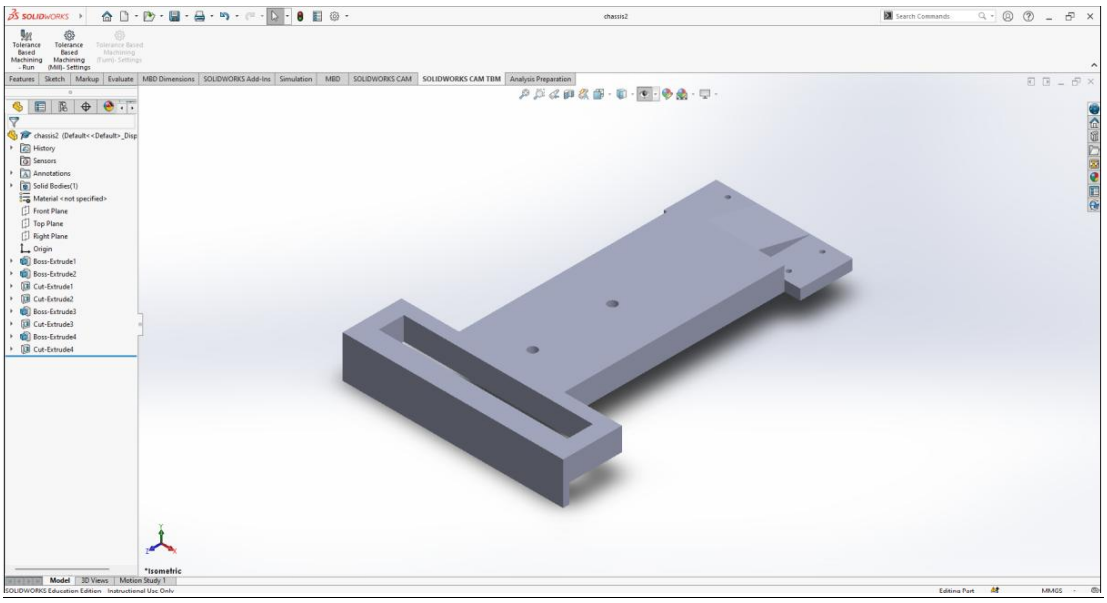


Figure 12: Final CAD Design

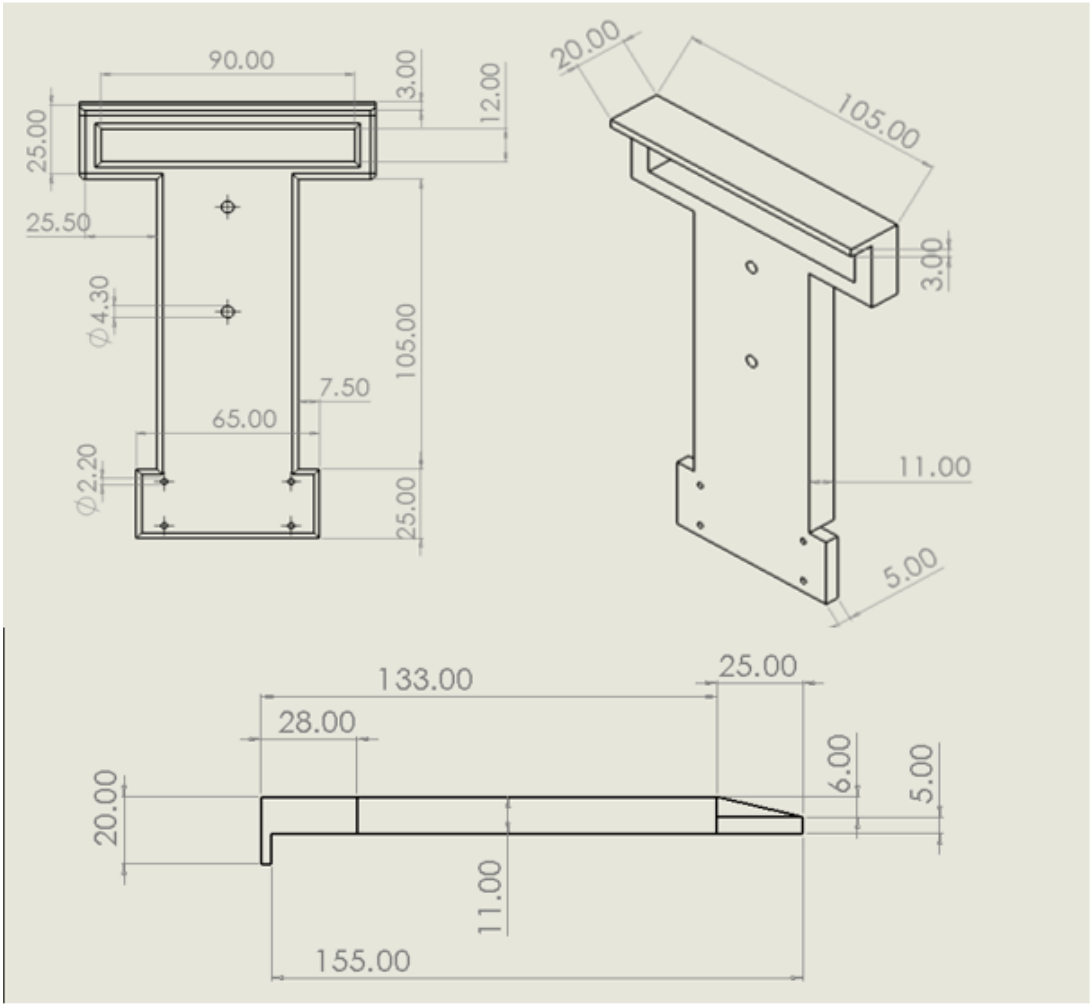


Figure 13: Final Engineering Drawing

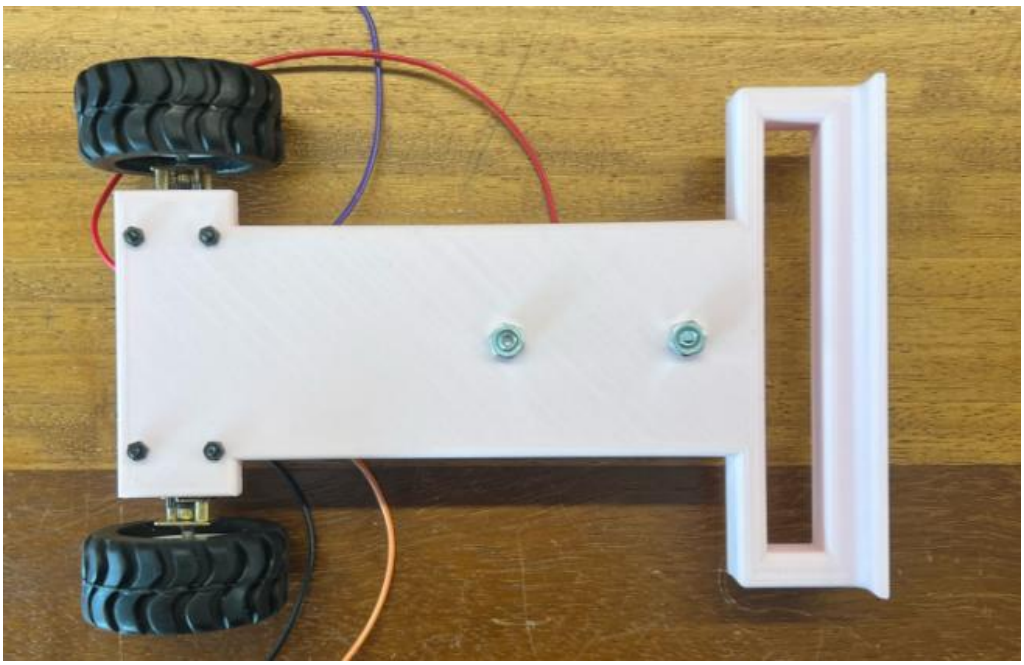


Figure 14: Birds Eye view of Final Chassis

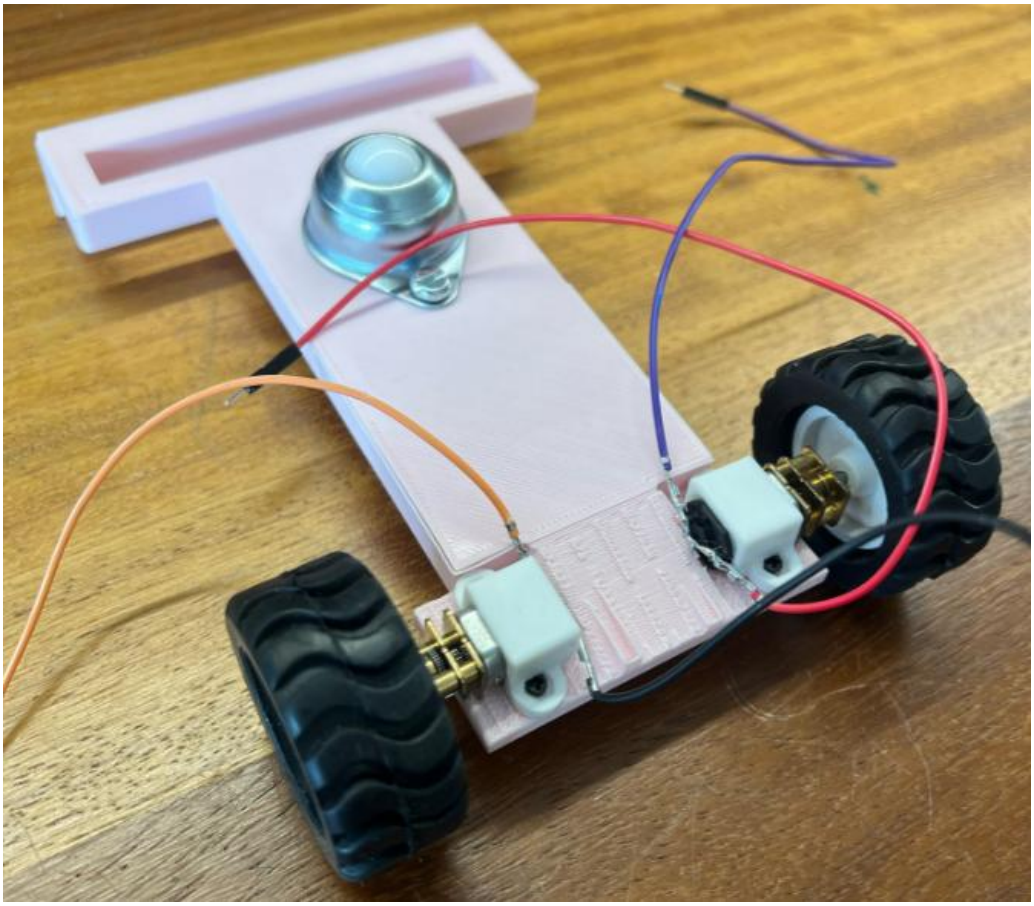


Figure 15: Bottom of the Chassis (Ramp between motors)

5.3. Coding and troubleshooting

After we have realised that using IR sensors was not an option considering this project, we have moved onto using RGB sensors. Due to their size, however, we were not able to use PID control as we couldn't fit enough sensors for this level of complexity. We were only limited to 3 sensors, so we decided that bang-bang was the best algorithm to use in this scenario.

The first issue encountered was that our minimum and maximum set values for the colour calibration were always inaccurate due to the different environments we had to test our robot in. To combat the issue, we have created a calibration function which would record a colour value every 25 microseconds, 80 times, and each time the minimum and maximum functions would be used to determine the new min/max value for that colour.

We would also initialise the values with the first pulse value before the loop began so that the initial value would not have been a garbage value. This would be able to mess

up the calibration if that value is greater than the recorded max or less than the recorded min.

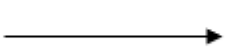
This function is then repeated for every sensor up until the sensor number reached is ‘SIZE’ which in this code is equal to 3. This is because our project only uses 3 different sensors.

```
void CalibrationBlue(int blueMin[], int blueMax[], int bluePW[], int So[]) {
  for (int k = 0; k < SIZE; k++) {
    // Initialize min and max with the first pulse value
    blueMin[k]= pulseIn(SO[k], LOW);
    blueMax[k] = blueMin[k];

    for (int i = 0; i < 80; i++) {
      bluePW[k]=pulseIn(SO[k], LOW);

      // Update min and max values
      blueMin[k]=min(blueMin[k], bluePW[k]);
      blueMax[k]= max(blueMax[k], bluePW[k]);

      delay(25);
    }
  }
}
```

The next issue encountered were the delays we were facing when testing the robot. There was a lot of lag between when the sensor values would detect the colour changes and when the motors would respond and change their speed/orientation.

The first change which we have made to reduce the lag was to reduce our threshold for the black colour detection. This would make the robot react much quicker to a colour change from white to black, therefore detecting the black line much sooner. We had to however keep this at a sensible value so that it would not accidentally confuse the black line with the white background. In the images below, bin0 represents the threshold below which white is detected and bin1 represents the threshold above which black is detected.

```
int bin0 = 120;
int bin1 = 450;
```



```
int bin0 = 20;
int bin1 = 50;
```

Although these changes had reduced the lag by a lot, there was still a considerable amount of lag remaining. After some more troubleshooting, we have decided to implement a ‘millis()’ function which would tell us how much time each function takes, hoping to improve whichever function takes the longest.

After implementing this function across our code, we have found out that the functions which are used to take the values coming from the sensors drastically increased our run time to around 89 milliseconds per cycle run.

To combat this issue, we decided to only implement one function instead of the previous three which would intake value for only one colour. After retesting with the new change, our runtime has drastically changed to around 4 milliseconds per cycle. We decided the colour which would be most suitable was blue as there was red writing on the paper and we did not want our sensors to react to it. Therefore, any colour between blue and green would have been sensible.

```
// Function to read Red Pulse Widths
void getRedPW(int redVal[], int redPW[], int redMin[], int redMax[]) { ---
}
// Function to read Green Pulse Widths
void getGreenPW(int greenVal[], int greenPW[], int greenMin[], int greenMax[ ]) { ---
}
// Function to read Blue Pulse Widths
void getBluePW(int blueVal[], int bluePW[], int blueMin[], int blueMax[]) { ---
}
void GetColour() {
   // Read Red Pulse Width
   getRedPW(redVal, redPW, redMin, redMax);

   // Read Green Pulse Width
   getGreenPW(greenVal, greenPW, greenMin, greenMax);

  // Read Blue Pulse Width
  getBluePW(blueVal, bluePW, blueMin, blueMax);

  // Print values
  char buffer[100];
  for(int i = 0; i < 3; i++) {
    sensVal[i] - redVal[i] + greenVal[i] + blueVal[i];
    sprintf(buffer, "Red PW %d = %3d | Green PW %d = %3d | Blue PW %d = %3d", i, redpW[i], i, greenPW[i], i, bluePW[i]);
    Serial.println(buffer);
  }
  sprintf(buffer, "Sensor 1 - %3d | Sensor 2 - %3d | Sensor 3 - %3d", sensVal[0], sensVal[1], sensVal[2]);
  Serial.println(buffer);
  Serial.println();
}
```

```
// Function to read Red Pulse Widths
void GetColour(int sensVal[], int bluePW[], int blueMin[], int blueMax[]) {

  // Set sensor to read Red only
  digitalWrite(S2,LOW);
  digitalWrite(S3,LOW);

  for(int i = 0; i < SIZE; i++) {
  // Read the output Pulse Width
  bluePW[i] = pulseIn(sO[i], LOW);
  //Map to limits
  sensVal[i] - map(bluePW[i], blueMin[i],blueMax[i],0,333);
  //Make sure value doesn't exceed limits
  if(sensVal[i] > 333) { sensVal[i] = 333;}
  if(sensVal[i] < 0) { sensVal[i] - 0;}
  }
```

Another issue encountered was the inconsistent line following and oversteering/understeering on a every turn. Since we first began the new bang-bang code by using a ‘switch’ statement, the robot would turn a set amount independently of the line. We also started by using only two sensors instead of the current three. This would only allow the robot to turn only a certain type of way when encountering turns and it would fail to turn sharp bends if it is able to turn shallow ones.

We have firstly switched from using a ‘switch’ statement to using caller functions. This would mean that while the sensors detected a certain pattern on the ground, the said function will be executed, however if that statement wouldn’t hold true anymore, we would check which other function can accommodate the new change and then call on that function. This process would happen in every single function which would enable for a much smoother line following experience and smoother turns.

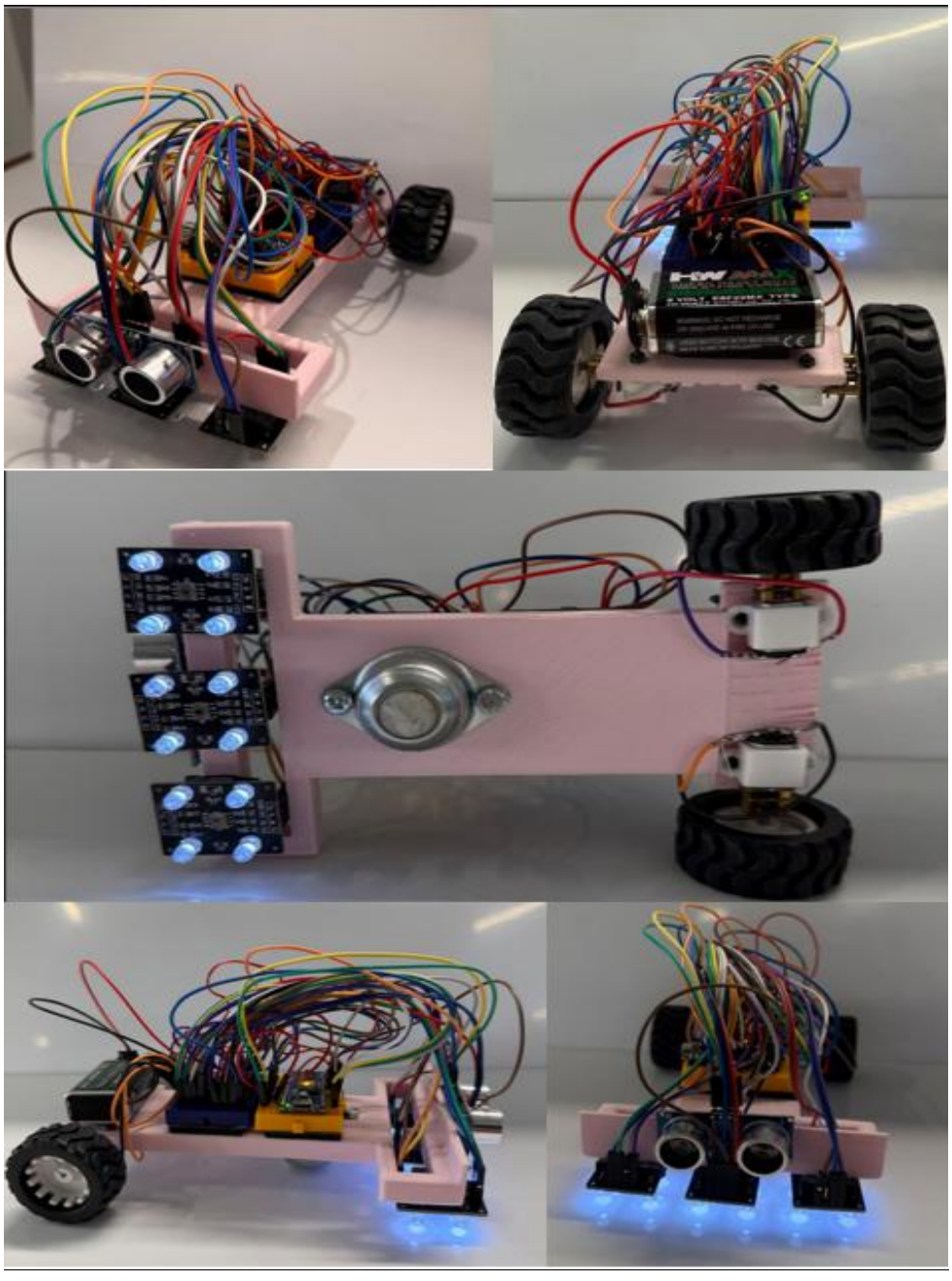
The next big change was adding an additional sensor in the middle. This would allow our car to oscillate less when following a straight line as it would have a reference sensor in the middle to help it. Sharp and shallow turns would now be possible. If the left sensor would be the only sensor detecting black, the robot would make a very shallow turn without reacting too much. If both the middle and left sensors would be the ones to detect black, the robot would now take a much more aggressive left turn until it detects that the line is straight or curving in another way.

```
int LineType = DetermineLine();
switch(LineType)
  case 0:
    Straight();
    break;
  case 1:
    RightTurn();
    break;
  case 2:
    LeftTurn();
    break;
  case 3:
    UTurn();
    break;
  default:
    Straight();
    break;
}
```

```
void Stop() { ···
}

void Straight() { ···
}

void RightTurn() { ···
}

void LeftTurn() { ···
}

void UTurn() { ···
}
int DetermineLine() {
  if(sensVal[0] < 100 && sensVal[1] > 900 && sensVal[2] < 100) {return 0;}
  if(sensVal[0] > 900 && sensVal[1] > 900 && sensVal[2] > 900) {return 0;}
  if(sensVal[0] < 100 && sensVal[1] > 900 && sensVal[2] > 900) {return 1;}
  if(sensVal[0] < 100 && sensVal[1] < 100 && sensVal[2] > 900) {return 1;}
  if(sensVal[0] > 900 && sensVal[1] > 900 && sensVal[2] < 100) {return 2;}
  if(sensVal[0] > 900 && sensVal[1] < 100 && sensVal[2] < 100) {return 2;}
  if(sensVal[0] < 100 && sensVal[1] < 100 && sensVal[2] < 100) {return 3;}
  return 0;
}
```

```
void Straight() {
  while((sensVal[0] < bin0 && sensVal[1] < bin0 && (sensVal[2] < bin0 || sensVal[2] > bin1)) || (sensVal[0] > bin1 && sensVal[1] > bin1 && sensVal[2] < bin0)) {
  int speedA = BaseSpeed;
  int speedB = BaseSpeed;
  Motor_Control(speedA, speedB);
  GetColour(sensVal, bluePW, blueMin, blueMax);
  }
  if(sensVal[0] > bin1 && sensVal[1] < bin0 && sensVal[2] < bin0) {
    LeftTurn();
  }
  if(sensVal[0] < bin0 && sensVal[1] > bin1 && sensVal[2] < bin0) {
    RightTurn();
  }
  if(sensVal[0] < bin0 && sensVal[1] > bin1 && sensVal[2] > bin1) {
    HardRight();
  }
  if(sensVal[0] > bin1 && sensVal[1] < bin0 && sensVal[2] > bin1) {
    HardLeft();
  }
}
```

5.4. Final Result (photos)



6. Evaluation and Discussion

Overview

In the evaluation phase, we focused on measuring how effectively our vehicle achieved the key design requirements set out in the beginning of the project. We approached this by constantly testing our prototype to assess its performance on reflective surfaces, obstacle detection accuracy, how it adapted to various differing tracks, and also its overall operational stability.

We implemented a hands-on and iterative approach to testing and refining our vehicle to ensure everything worked correctly. This meant redesigning our chassis multiple times, testing the robot on both reflective and non-reflective surfaces, and carefully calibrating our sensors. Each round of testing helped us identify and solve new issues, giving us a deeper understanding of how robots behaved at every stage of development.

Below is a table that briefly outlines how we evaluated each key design requirement:

Design Requirement	**Aim and Target value**	**Testing method**
Line-following accuracy	Constant tracking of black line on a glossy surface	Testing our RGB sensors on a reflective surface
Obstacle detection and avoidance	Be able to detect an obstacle and move around it then continue on track	Testing the range using our ultrasonic sensor with stationary objects
Precision of movement	Be able to take sharp turns and adjust itself accordingly	Behavioral analysis via Bang-Bang control algorithm
Structural stability of robot	Remain intact despite repeated use and testing	Applying physical pressure to robot to test its durability
Remaining within budget	<£100	Maintaining and updating a strict total cost and order sheet
Ease of use and autonomous ability	Robot must require minimal user input to function	Tried implementing the start button, however for more simplicity we connected the wire from the battery directly to the VCC pin of the Arduino

Prototype

Our final prototype was a compact Arduino-based robot with a lightweight and stable 3D-printed chassis that was designed to efficiently support all the components, including the many wires and mounting points for sensors and motors. The vehicle also featured RGB colour sensors and an ultrasonic sensor which enabled it to autonomously follow a black line on a glossy surface (RGB sensors) and be able to detect obstacles (ultrasonic sensors), which were the key design requirements set out in the beginning of the project.

The key elements in the robot included:

- 3x RGB colour sensors: These were used to detect the colour contract between the black track and its white background, on the reflective surface.
- Ultrasonic sensor (HC-SR04): Our ultrasonic sensor was placed at the front of the vehicle and was used to detect and avoid obstacles in its path.
- 2x DC Motors with motor driver (TB6612FNG): The motor driver and motors enable the robot to move.
- Arduino Nano: The Arduino Nano was our microcontroller used to process the various sensor inputs and control the vehicle's movement in real-time.
- Custom 3D-printed chassis: This was designed to optimize the efficiency of our robot, including the positioning of our components, its weight and maneuverability.

Discussion of our tests and results

1. Line-Following Accuracy

Introduction: The robot must use RGB colour sensors to accurately follow a black line on a white background, with the surface being reflective.

Method: We repeatedly tested the robot on the simple oval shaped track assigned to us and also on the Silverstone track. We also implemented the Bang-Bang control algorithm to determine the robot's direction based on the input from the sensor.

Results: The vehicle could successfully follow the black line under most conditions, and it performed well on wide curves and straights. However, it'd occasionally struggle with taking sharp turns which may be due to the limited responsiveness of the Bang-Bang system compared to PID.

Discussion: Although the system worked well for simple line-following paths, it could use some further enhancements to better handle sharp turns, such as a smoother control algorithm e.g. PID.

2. Obstacle Detection and Avoidance

Introduction: The vehicle must be able to detect obstacles in its path and avoid them accordingly.

Method: We attempted to use the ultrasonic sensor to detect obstacles and send the data to the Arduino, which would enable the robot to stop and move around the obstacle, then continue its path.

Results: The robot could successfully stop when an obstacle is placed in front of it, however it was unable to move around the obstacle and continue its path. Also, its response to halting was inconsistent, as sometimes it would just continue moving, and go through the obstacle.

Discussion: The detection system works; however, it still has room for improvement. To improve responsiveness, we could maybe implement a second ultrasonic sensor to aid the vehicle's decision making.

3. Control System Response

Introduction: We had to implement an effective control system that adjusted the motor speed and the vehicle's direction based on the input it received from the sensor.

Method: We implemented a Bang-Bang control system since it was simpler than PID, and it also required less sensors, and given our chassis size, we were unable to fit more colour sensors.

Results: The vehicle was able to follow and adjust itself according to the path laid out to it, however its movement was slightly erratic in some parts of the track due to the binary nature of Bang-Bang control.

Discussion: Although the Bang-Bang system proved to be effective for initial testing, it lacks the smoother movement of a PID-based control system, and this would correlate to our project since some of the sharp turns in the track require more smooth and efficient movement.

4. Chassis and Structural Integrity

Introduction: The chassis must be strong enough to support all components and be designed to place components efficiently, while also withstanding movement while testing and mild impacts i.e. with the obstacle.

Method: We applied physical pressure to the chassis to assess its stability, and we also experimented with different chassis designs.

Results: The final 3D-printed design which included varying thickness throughout the chassis, a support ramp for the motors and a larger hole for wires proved to be more stable and efficient than our previous designs. The previous designs included our laser-cut wood design, which was completely unstable as the vehicle wasn't level, didn't have any holes for wires and wasn't designed with efficiency overall. Our first 3D printed design was good however it was unstable and broke at the intersection between the chassis and the motors, therefore forcing us to add the support ramp, which was explained in further detail during our design section of the report. Additionally, we believe we could've made the chassis smaller in length in the future. This is because the long length of the chassis may have been one of the reasons it was unable to take sharp turns as it increased the vehicle's turning radius, making quick directional changes more difficult.

5. Budget

Introduction: Our project was capped at a total expenditure of £100

Method: We kept a detailed cost spreadsheet which we updated and included the costs of all components and materials used in the final prototype.

Results: We successfully built the vehicle with an expenditure of approximately £75, staying well below our budget.

Discussion: Our cost-efficient strategies such as keeping the cost very low from the early planning stages allowed us to be more flexible later in our project and not worry about going over the limit. We also utilized our universities facilities such as the 3D printer which saved the cost of buying a chassis or paying to use a 3D printer.

Assessment

Overall, our vehicle met majority of the design requirements, meaning the vehicle followed the black line on the reflective surface to an extent however struggling on very sharp turns and bends, detected obstacles to an extent, and was also structurally sound.

However, the project did have its limitations and areas for improvement. This included:

- Its turning performance, which isn't great on sharp turns, may be due to the simplicity of the Bang-Bang algorithm and the length of the chassis, as previously mentioned.
- Also, the robot doesn't move around and avoids an obstacle on its path, it only halts when an obstacle is front of it, or will continue moving, which is another area for refinement.

That said, the iterative improvements we made throughout our project, such as redesigning the chassis multiple times, and switching control algorithms from PID to Bang-Bang, demonstrates our team's problem-solving skills, and our commitment to building a low-cost, stable and efficient autonomous line-following robot.

7. Future Improvements

After our results and tests there are still some issues that can be addressed and fixed for any future improvements. The aspects of the robot that can be changed is slightly refining the chassis design again, returning to PID control System and debugging the problems encountered in ultrasonic sensor code.

Chassis Improvements

- After many tests, the robot chassis was still slightly large but this time the length of the robot from front to rear was just slightly large causing issues in the turning of the robot and struggling on the extremely aggressive turns on the track.
- The large rectangular cutout at the front couldn't fully house 3 RGB sensors properly and we couldn't add a fourth sensor for even more sensitivity.
- Therefore, to solve these issues, the length can be reduced of the chassis by making the midsection shorter. Make the front slightly wider for a larger rectangular cutout to accommodate these RGB sensors.

I will design a proposed chassis on the Inkscape with all the required adjustments, which can be used for a future project:

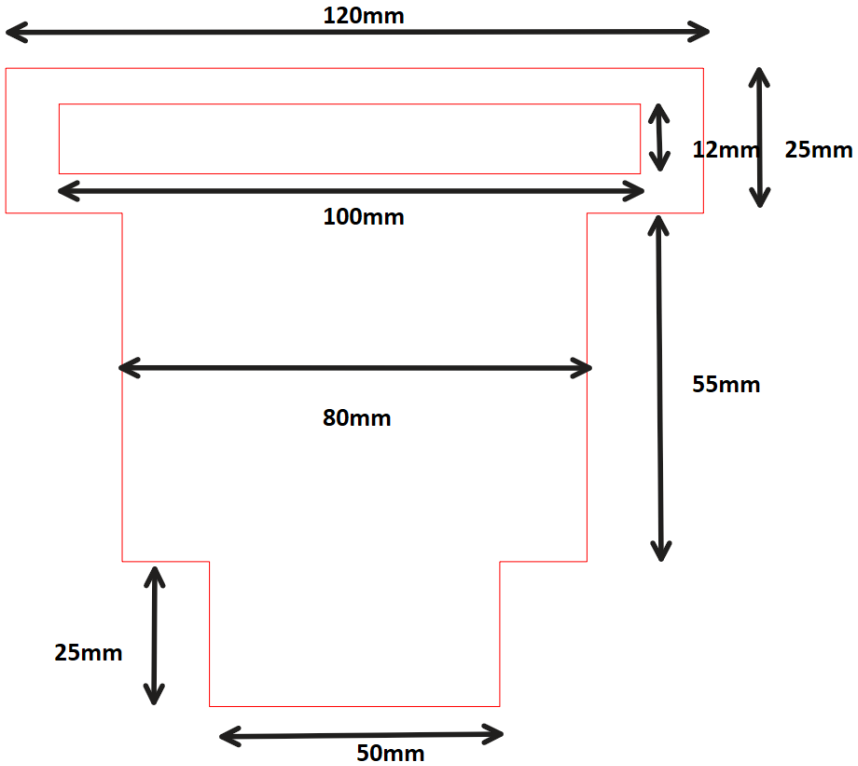


Figure 16: Proposed Chassis design for future

Returning to PID control System

Returning to PID (Potential, Derivative, Integral) for the future will enhance the navigation of the robot making it respond better to the line position and making adequate and quick adjustment to stay on track. To have a good PID control system more sensors are needed to maintain high detection resolution, so considering the need for more sensors it means they also need to be small enough to fit many of them across the chassis. The most suitable sensor is the TCS34725 RGB sensor. Integrating minimum 5 of these sensors will support the PID system much better allowing for better robot performance.

Debugging the Ultrasonic sensor

Addressing the ultrasonic sensor functionality is very important, these current issues in the sensor could stem from incorrect wiring but also issue in the coding. More consistent and systematic debugging and vigorous testing can solve these issues.

8. Conclusion

We were set this ambitious project in the beginning of the semester with the goal of developing a fully autonomous vehicle capable of accurately following a black line on a white background, with a reflective surface. The robot also had to be able to detect and avoid obstacles - all within a strict £100 budget, and we are pleased to say that the project has been a success. Due to constant iteration, intricate design and plenty of hands-on testing, we were able to build a working vehicle that follows a defined path using RGB colour sensors and can detect obstacles using an ultrasonic sensor.

We faced and overcame many challenges throughout the development process. Some of these challenges we faced included choosing the correct type of sensors, adjusting our control algorithm, and redesigning our chassis several times in order to provide maximal structural support and maneuverability. Our final design implements a Bang-Bang control system which has proven to be stable and reliable for our robot, especially when it's paired with our custom 3D printed chassis that was designed to prioritize efficient wiring and clean sensor placement.

However, full obstacle avoidance still hasn't been implemented for our robot yet. This is because whenever the vehicle detects an obstacle, it either halts or continues to move, depending on its position and timing. Also, the robot still slightly struggles with very sharp turns, like those included on the Silverstone track which we were tested upon. Although robots have some faults, we believe these points provide a guide for future refinement and a functional foundation for us to build upon.

Overall, we were successful in completing this project and created a low-cost, sensor-integrated autonomous vehicle that displays line following capabilities and basic obstacle detection responsiveness. This project doesn't only reflect our technical achievement, but also our ability to collaborate with ideas and work as a team, whilst also being able to troubleshoot and adapt our robot when we face any issues. We are proud of what we have built and are excited to see what future holds for autonomous vehicles.

9. References

[1] Arduino, "Arduino Uno Rev3 Datasheet," Arduino.cc, 2021. [Online]. Available: https://docs.arduino.cc/hardware/uno-rev3. [Accessed Apr. 8, 2025].

[2] Arduino, "Arduino Nano Datasheet," Arduino.cc, 2021. [Online]. Available: https://docs.arduino.cc/hardware/nano. [Accessed Apr. 8, 2025].

[3] AMS, "TCS3200 Programmable Color Light-to-Frequency Converter Datasheet," AMS, 2020.

[4] A. Carullo and M. Parvis, "An ultrasonic sensor for distance measurement in automotive applications," IEEE Sensors J., vol. 1, no. 2, pp. 143-147, Aug. 2001.

[5] STMicroelectronics, "L298 Dual Full-Bridge Driver Datasheet," STMicroelectronics, 2000.

[6] Toshiba, "TB6612FNG Dual Motor Driver IC Datasheet," Toshiba Electronics Devices & Storage Corporation, 2020.

[7] P. Corke, Robotics, Vision and Control: Fundamental Algorithms in MATLAB, 2nd ed., Springer, 2017, ch. 4, pp. 105-110.

[8] S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics, Cambridge, MA: MIT Press, 2005, ch. 3.

[9] Hitec, "HS-422 Servo Motor Datasheet," Hitec RCD USA, Inc., 2020.

[10] J. A. Castellanos, J. Neira, and J. D. Tardos, "Limits to the consistency of EKF-based SLAM," in IFAC Proc. Volumes, vol. 37, no. 8, pp. 716-721, 2004.